

ID Layers in GnuCash (Users and Programmers)

Overview

Generally speaking, you normally have two ID layers in a software, i.e. two layers to identify objects:

- the technical layer
- the business-logic layer

A user normally only deals with the business-logic layer, whereas a programmer would typically work primarily with the technical layer and provide services for the business-logic layer only indirectly, by translating between it and the technical layer.

In GnuCash, this system is used for all entities but one: the securities.

Security IDs in GnuCash

Figure 1 shows the two levels of security IDs in GnuCash.

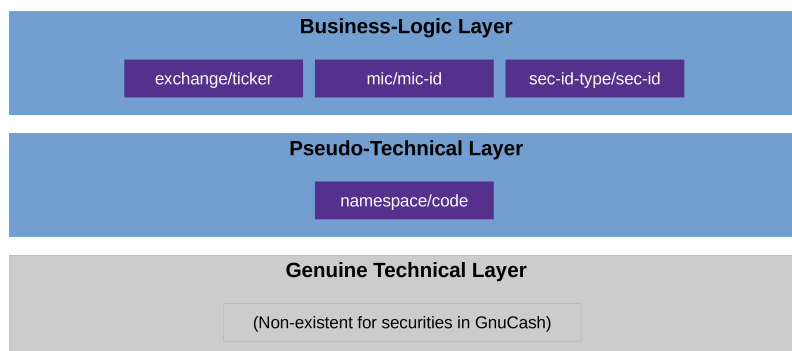


Figure 1: ID layers in GnuCash

In theory, the two layers mentioned in the previous section (i.e., the lowest and the highest one in figure 1) are clearly separated, i.e. technical IDs and business-logic ID do not have anything to do with each other. However, the GnuCash developers chose to make things a little more

complicated: They intentionally blurred the line between the two layers, so that technical IDs are, in fact, pseudo-technical and quasi-business.

If they had chosen to treat securities as any other entity in GnuCash, then a security's technical ID would look something like this: `14305dc80e034834b3f531696d81b493`. This string obviously has no meaning, i.e. it cannot not be “interpreted” or “understood” nor is it meant to be.

Well, they chose another approach: In GnuCash, a (pseudo-)technical ID looks something like these:

- `EURONEXT:SAP`
- `ISIN:DE000BASF111AP`

Just by glancing at these, you can see that they are essentially business-logic IDs masquerading as technical ones; they do mean something (i.e., they have semantics) and can thus very well be interpreted and understood.

It is important to understand this (non-)difference between technical and business-logic IDs in GnuCash before reading the other documents; they might otherwise confuse you.

You can also see that the two examples above are composed of two parts, which is no coincidence but rather the system that GnuCash imposes: `<namespace>:<code>`. Whereas the name space can, in theory, be freely chosen by the user, in practice it makes sense *not* to do so but instead to choose from a pre-defined set that `JGnuCashLib` provides. This then leads to the concept of providing IDs “indirectly”, i.e. composing them: providing the name space and the code separately.